

LAB 4 - SYNCHRONIZATION: RISPOSTE

Risposta #1

- Perché il vero ciclo di busy waiting sarà effettuato da `spinlock_data_get`, meno dispendioso a livello di impatto sul sistema, rispetto a `spinlock_data_testandset`. Un loop su `spinlock_data_get` da solo non può essere fatto poiché non effettua test-andset atomico.
- No, per quanto riguarda `spinlock_data_testandset`, `spinlock_acquire` esce dal loop correttamente quando la funzione ritorna 0, mentre `spinlock_acquire2` cicla indefinitamente. Il `continue` deve essere sostituito con un `break`.

Risposta #2

Il lock è una primitiva di sincronizzazione che realizza mutua esclusione in non busy waiting. Introduce il concetto di ownership, ovvero solo il thread che ha acquisito il lock può rilasciarlo. Il semaforo è anche esso una primitiva di sincronizzazione dove l'accesso alla sezione critica è regolata dalla condizione che il counter del semaforo (inizializzato ad un valore arbitrario alla sua creazione) sia positivo. Quando un thread/processo richiede l'accesso alla S.C. effettuando una wait, il contatore viene decrementato mentre all'uscita dalla sezione critica il contatore viene aumentato effettuando una signal.

Risposta #3

La differenza fondamentale tra spinlock e lock è la realizzazione di busy waiting. Nel primo, infatti il thread richiedente l'accesso non è bloccato mentre nel secondo sì. Entrambi richiedono l'ownership (concetto spiegato prima). Solitamente, è preferibile l'utilizzo degli spinlock quando le S.C sono ridotte e quindi è preferibile evitare continui cambi di contesto dei thread.